győri szakképzési centrum

Jedlik Ányos
Gépipari és Informatikai
Technikum és Kollégium

9021 Győr, Szent István út 7.
+36 (96) 529-480
+36 (96) 529-448

OM: 203037/003
jedlik@jedlik.eu
www.jedlik.eu

Záródolgozat feladatkiírás

Tanuló(k) neve: Anda Kristóf, Bicskey Balázs, Borbély Dominik Péter

Képzés: nappali munkarend

Szak: 5 0613 12 03 Szoftver fejlesztő és -tesztelő technikus

A záródolgozat címe:

„SkillTree”

Konzulens: Kottra Richárd

Beadási határidő: 2026. 04. 20.

Győr, 2026. április 20.

Módos Gábor
Igazgató

Konzultációs lap

	A konzultáció		Konzuléns aláírása
	ideje	témája	
1.	2026.02.15.	Témaválasztás és specifikáció	
2.	2026.03.14.	Záródolgozat készültségi fokának értékelése	
3.	2026.04.20.	Dokuméntáció véglégesítése	

Tulajdonosi nyilatkozat

Ez a dolgozat a saját munkánk eredménye. Dolgozatunk azon részeit, melyekét más szerzők munkájából vettünk át, egyértelműen megjelöltük.

Ha kiderülne, hogy ez a nyilatkozat valótlan, tudomásul vesszük, hogy a szakmai vizsgabizottság a szakmai vizsgáról kizár minket és szakmai vizsgát csak új záródolgózat készítése után tehetünk.

Győr, 2026. április 20.

Tanulók aláírása:

Anda Kristóf

Bicskey Balázs

Borbély Dominik Péter

Tartalomjegyzék

1. Bevezetés 5
1.1 Projekt rövid bemutatása 5
1.2 Használt technológiák 5
2. Csapatmunka 6
2.1 Feladatok elosztása 6
2.2 Github használata 6
3. Backend 9
3.1 Funkció lista 9
3.2 Elérési utak 9
3.3 Modell osztályok 16
3.4 Adatbázis létrehozás és módosítás: migrációk 17
3.5 Adatbázis biztonságos elérése 18
3.6 Kontroller osztály 18
3.7 Adatok validálása 20
3.8 Minta adatok generálása 20
3.9 Adatbázis 22
4 Frontend 24
4.1 Bevezetés 24
4.2 Frontend Architektúra és Kliens-Szerver Kapcsolat 24
4.3 Adatlekérés (Data Fetching) és API Kommunikáció 26
4.4 Munkamenet-kezelés és Hitelesítés 28
4.5 Felhasználói felület és komponenskönyvtár 29
4.6 Funkcionális modulok: Kurzuskezelés és Tartalomkezelés 30
4.7 Adminisztrációs felület és Jogosultságkezelés 32
4.8 Közösségi tér és Valós idejű kommunikáció (Community Hub) 34
5 Tesztelés 36
5.1 Backend tesztelés 36
5.2 Frontend Tesztelés 37
6 Deployment 38
6.1 Docker konténerizálás 38
6.2 Projekt futtatása lokálisan 38

6.3 AWS S3 és Cloudflare R2 használata 38

7 Felhasználói útmutató 40

8 Irodalomjegyzék 41

1. Bevezetés

1.1 Projekt rövid bemutatása

A SkillTree egy modern webes e-learning alkalmazás, amely az online tudásmegosztás és tanulás folyamatát hivatott megkönnyíteni azzal, hogy egységesen elérhetővé tesz minden olyan funkciót, amely a kurzusok készítéséhez, kezeléséhez és az azokon való részvételhez szükséges. Így tehát az alkalmazás a tartalomgyártás egyszerűsítését, a diákok fejlődésének követését, és az interaktív felhasználói élmény biztosítását hivatott lehetővé tenni egy mindenki számára intuitív módon. Célja, hogy az online oktatás folyamatát gördülékenyebbé tegye, emellett a felhasználók számára nyújtson egy áttekinthető módot arra, hogy naprakészek legyenek a tanulmányaikkal.
A platform így az alábbi kulcsfontosságú igényekre és funkciókra fókuszál:

- **Szerepkör-alapú jogosultságkezelés:** A rendszer többféle felhasználói szintet különböztet meg (alap felhasználó, adminisztrátor), így biztosítva, hogy mindenki a saját jogosultságainak megfelelő, letisztult felülettel és eszközkészlettel találkozzon.
- **Interaktív tanulói élmény:** Az alap felhasználók egy könnyen kezelhető felületen böngészhetnek a feltöltött kurzusok között (például név, kategória és egyéb szűrők alapján), egyszerűen jelentkezhetnek azokra, és nyomon követhetik a saját haladásukat.
- **Komplex kurzuskészítő felület:** A felhasználók számára a rendszer egy dedikált felületet biztosít, ahol új kurzusokat hozhatnak létre, menedzselhetik a már meglévő tananyagaikat, és egyetlen kattintással kiadhatják azokat a közönség számára.
- **Beépített kommunikációs csatornák:** A platform nagy hangsúlyt fektet a közösségi élményre; a beépített chat funkció segítségével a felhasználók személyesen vagy csoportosan is cseveghetnek, ami közvetlen kapcsolatfelvételt tesz lehetővé a diákok és a kurzusok készítői között.

1.2 Használt technológiák

A technológiák kiválasztásakor a legfőbb szempont a magas teljesítmény, a kiváló fejlesztői élmény és a költséghatékony skálázhatóság volt.
Verziókövetés: Git, Github
Backend: Laravel
Frontend: Next js
Adatbázis: PostgreSQL
Objektumtárolás (Képek/Videók): Amazon S3, Cloudflare R2

2. Csapatmunka

A hatékony csapatmunka érdekében fontos volt az egyértelmű feladatmegosztás. Tudtuk, hogy mind egymás, mind tanáraink révén tudunk segítséget kérni, ha valamilyen probléma merül fel a fejlesztés során.

2.1 Feladatok elosztása

A projektünk felhasználói felületét (frontend) Anda Kristóf valósította meg, a backend fejlesztéséért Borbély Dominik Péter felelt, míg a tesztelési folyamatokat, az üzemeltetést (deployment), valamint a felhőalapú kép- és videófeltöltés infrastruktúrájának integrációját Bicskey Balázs végezte.

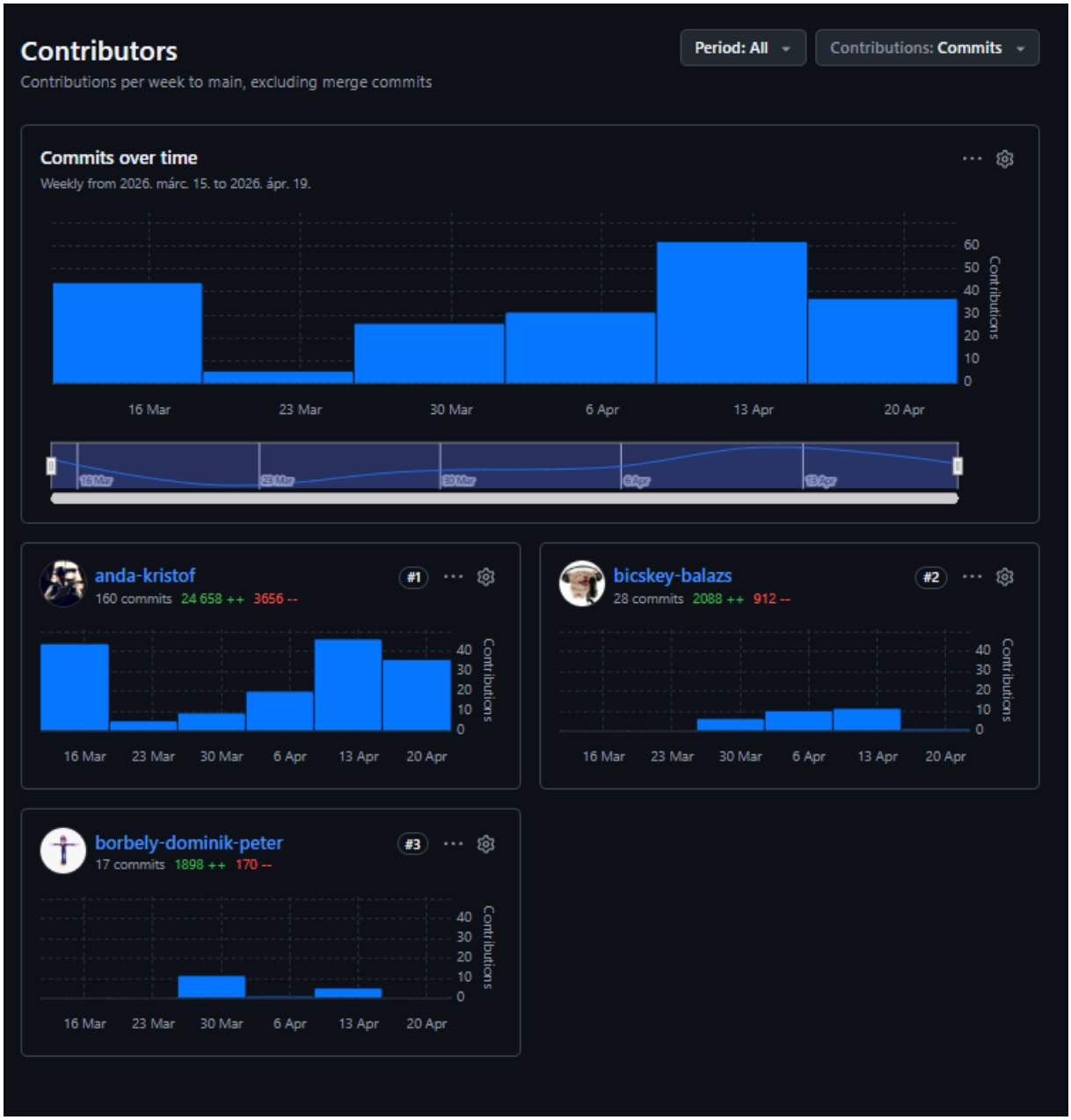
A feladatok szigorú, teljes mértékű szétválasztása azonban a mi esetünkben is lassította volna a fejlesztési folyamatot. Éppen ezért a munkánk során folyamatosan segítettük egymást, és a szoros határidők vagy az összetettebb funkciók miatt mindenki bekapcsolódott a projekt más szegmenseibe is.

A tervezési fázisban és az adatbázis struktúrájának kialakításában mindhárman közösen vettünk részt, mivel ezek az alapvető architekturális döntések a rendszer egészének működését meghatározzák. A fejlesztés során kiemelt figyelmet fordítottunk a minőségbiztosításra: a dedikált tesztelési és üzemeltetési feladatok garantálták, hogy a frontend és a backend zökkenőmentesen kommunikáljon, a funkciók stabilan működjenek, és a rendszer hibamentesen álljon fel az éles, felhőalapú környezetben.

2.2 Github használata

A kollaboráció és a forráskód kezelése GitHub alapokon valósult meg. A választás elsődleges szempontja a csapat egységes technológiai ismerete volt: az oktatás során szerzett többéves rutin lehetővé tette a verziókövetés, valamint a feladatkezelő (issue tracking) funkciók azonnali, hatékony alkalmazását.

Képek a github statisztikákról (frontend és backend):


Contributors
Contributions per week to main, excluding merge commits
Period: All
Contributions: Commits
Commits over time
Weekly from 2026. márc. 15. to 2026. ápr. 19.
60
50
40
30
20
10
0
Contributions
16 Mar
23 Mar
30 Mar
6 Apr
13 Apr
20 Apr
anda-kristof
160 commits 24 658 ++ 3656 --
#1
bicskey-balazs
28 commits 2088 ++ 912 --
#2
borbely-dominik-peter
17 commits 1898 ++ 170 --
#3

(frontend)

Contributors
Contributions per week to main, excluding merge commits
Period: All
Contributions: Commits
Commits over time
Weekly from 2026. febr. 15. to 2026. ápr. 19.
80
60
40
20
0
Contributions
16 Feb
23 Feb
2 Mar
9 Mar
16 Mar
23 Mar
30 Mar
6 Apr
13 Apr
20 Apr
borbely-dominik-peter
#1
288 commits 24 300 ++ 3963 --
50
25
0
Contributions
23 Feb
9 Mar
23 Mar
6 Apr
20 Apr
bicskey-balazs
#2
90 commits 11 326 ++ 9895 --
50
25
0
Contributions
23 Feb
9 Mar
23 Mar
6 Apr
20 Apr

(backend)

3. Backend

Az alkalmazás backend-je a Laravel PHP alapú keretrendszer, jelenlegi(2026-03-05) legújabb, 12-es verziójában készült.
Egy modern backend követelményeit a legtöbb esetben a Laravel-be beépített funkciók szolgálják ki.
Ezekre, a funkciók kifejtésénél bővebben kitérünk.

3.1 Funkció lista

- Modellosztályok: Laravel Eloquent model
- Kontrollerosztályok: Laravel Controller
- Adatok validálása: Laravel Request
- Hamis teszt adatok létrehozása: FakerPHP/Laravel Seeder
- Online dokumentáció: Dedoc Scramble

3.2 Elérési utak

A backend 100-nál is több elérési úttal rendelkezik, ezért ebben a dokumentációban csak néhány fontosabb elérési utat emelünk ki.
Az elérési utakat az api.php definiálja, a beépített Route::apiResource() metódus segítségével, ami a CRUD műveletek elérési útjait automatikusan létrehozza.
Az összes elérési út online dokumentációja több URL-en is elérhető, a futtatási módnak megfelelően.
http://127.0.0.1:8000/docs/api/ -> Lokális futtatás
http://127.0.0.1:9998/docs/api/ -> Docker konténeres futtatás
https://skillstree-vizsgaremek-backend.jcloud.jedlik.cloud/docs/api -> Távoli elérés

3.2.1 Elérési utak: /api/courses/{id}

Az elérési út lekérdezi a paraméterben megkapott „id” azonosítóval rendelkező kurzus összes adatát.

- Metódus: GET
- Válasz: az {id} paraméterben megadott id-val rendelkező kurzus összes adata
- Státuszkódok: - 200 -> lekérés sikeres
  - 401 -> jogosultság nincs
  - 404 -> nem található ezzel az id-val kurzus
- Válasz minta:

```
{
    "id": 1,
    "course_name": "At ad provident quae tempore quo.",
    "created_at": "2026-03-02 19:37:01",
    "updated_at": "2026-03-02 19:37:01",
    "category_id": 1,
    "owner_id": 1,
    "max_level": 50938,
    "membercount": 21893,
```

```
        "price": 4909,
        "primary_color": "#000000",
        "secondary_color": "#000000",
        "description": "Sed id molestias.",
        "preview_video_url": "Sit porro aspernatur aut.",
        "course_logo": "Exercitationem aut qui dolorem. ",
        "banner_img": "A adipisci alias minima dolorem. ",
        "design_img": "Incidunt vitae earum inventore eos.",
        "side_img": "Quaerat suscipit molestiae aspernatur …",
        "pay_interval": "Corrupti eum."
    }
```

3.2.2 Elérési utak: /api/users/{id}

Az elérési út lekérdezi a paraméterben megkapott „id” azonosítóval rendelkező felhasználó összes adatát.

- Metódus: GET
- Válasz: az {id} paraméterben megadott id-val rendelkező felhasználó összes adata
- Státuszkódok: - 200 -> lekérés sikeres
  - 401 -> jogosultság nincs
  - 404 -> nem található ezzel az id-val felhasználó
- Válasz minta:

```
    {
        "id": 1,
        "name": "igjwpo2",
        "email": "f@d.h",
        "email_verified_at": null,
        "password": "$2y$12$fasdfghjkrtzuioxcvbnm",
        "remember_token": null,
        "created_at": "2026-03-05 09:37:16",
        "updated_at": "2026-03-05 09:37:16",
        "profile_picture": "asdlkjlakdjslks",
        "isadmin": true,
        "coursecreatenumber": 0
    }
```

3.2.3 Elérési utak: /api/courses

Az elérési út lekérdezi a paraméterben megkapott „id” azonosítóval rendelkező kurzus összes adatát.

- Metódus: POST
- Request minta:

```
    {
        "id": 0,
        "course_name": "string",
        "created_at": "2019-08-24T14:15:22Z",
        "updated_at": "2019-08-24T14:15:22Z",
```

```
        "category_id": 0,
        "owner_id": 0,
        "max_level": 0,
        "membercount": 0,
        "price": "string",
        "primary_color": "string",
        "secondary_color": "string",
        "description": "string",
        "preview_video_url": "string",
        "course_logo": "string",
        "banner_img": "string",
        "design_img": "string",
        "side_img": "string",
        "pay_interval": "string"
    }
```

- Válasz: az új kurzus adatai
- Státuszkódok: - 200 -> sikeres
    - 401 -> jogosultság nincs
    - 422 → Hibás request
- Válasz minta:

```
    {
        "id": 0,
        "course_name": "string",
        "created_at": "2019-08-24T14:15:22Z",
        "updated_at": "2019-08-24T14:15:22Z",
        "category_id": 0,
        "owner_id": 0,
        "max_level": 0,
        "membercount": 0,
        "price": "string",
        "primary_color": "string",
        "secondary_color": "string",
        "description": "string",
        "preview_video_url": "string",
        "course_logo": "string",
        "banner_img": "string",
        "design_img": "string",
        "side_img": "string",
        "pay_interval": "string"
    }
```

3.2.4 Elérési utak: /api/login

Az elérési út a request adatok alapján bejelentkeztet egy már létező felhasználót.

- Metódus: POST
- Request minta:

```
{
        "profile_picture": "string",
        "password": "string",
        "email": "user@example.com",
        "name": "string",
        "isadmin": true,
        "coursecreatenumber": 0
}
```

- Válasz: a felhasználó adatai + új token / vagy hibaüzenet
- Státuszkódok: - 200 -> sikeres
    - 401 -> jogosultság nincs
    - 422 -> a request-body adatai helytelenek
    - Válasz minta:

```
{
        "user": {
                "id": 0,
                "name": "string",
                "email": "string",
                "email_verified_at": "2019-08-24T14:15:22Z",
                "created_at": "2019-08-24T14:15:22Z",
                "updated_at": "2019-08-24T14:15:22Z",
                "profile_picture": "string",
                "isadmin": "string",
                "coursecreatenumber": 0
        },
        "token": "string"
}
```

- Egyéb: A token-t a Laravel automatikusan generálja

3.2.5 Elérési utak: /api/register

Az elérési út a request adatok alapján létrehoz egy új felhasználót, majd bejelentkezteti(tokent generál).
- Metódus: POST
- Request minta:

```
{
        "password": "string",
        "email": "user@example.com",
        "name": "string",
        "isadmin": true,
        "coursecreatenumber": 0
}
```

- Válasz: az új felhasználó adatai + token
- Státuszkódok: - 200 -> sikeres
    - 422 -> a request-body adatai helytelenek

- Válasz minta:

```
{
        "user": {
                "id": 0,
                "name": "string",
                "email": "string",
                "email_verified_at": "2019-08-24T14:15:22Z",
                "created_at": "2019-08-24T14:15:22Z",
                "updated_at": "2019-08-24T14:15:22Z",
                "profile_picture": "string",
                "isadmin": "string",
                "coursecreatenumber": 0
        },
        "token": "string"
}
```

- Egyéb: A token-t a Laravel automatikusan generálja
  - Az image-t a frontend adja át bináris fájlként

3.2.6 Elérési utak: /api/logout

Az elérési út kijelentkeztett egy bejelentkezett felhasználót.

- Metódus: POST
- Request minta: üres request
- Válasz: sikeres kijelentkezés / autentikációs hiba
- Státuszkódok: - 200 -> sikeres
  - 401 -> autentikációs hiba
- Válasz minta:

```
{
"message": "Successfully logged out and token has been deleted."
}
```

3.2.7 Elérési utak: /api/lessons

Az elérési út egy új leckét hoz létre a request adatai alapján.

- Metódus: POST
- Válasz: az új lecke adatai / autentikációs hiba / request hiba
- Request minta: az id kivételével megegyezik a válasszal.
- Státuszkódok: - 200 -> sikeres
  - 401 -> autentikációs hiba
  - 422 -> request body hiba

- Válasz minta:

```
{
        "id": 0,
        "index": 0,
        "subcourse_id": 0,
        "name": "string",
```

```
        "description": "string",
        "xp": 0,
        "created_at": "2019-08-24T14:15:22Z",
        "updated_at": "2019-08-24T14:15:22Z"
    }
```

3.2.8 Elérési utak: /api/multichoice_questions/{id}

Az elérési út lekérdezi a paraméterben megkapott „id” azonosítóval rendelkező felelet-választós kérdés összes adatát.

- Metódus: GET
- Válasz: Az id azonosítójú kérdés adatai / request hiba
- Státuszkódok: - 200 -> sikeres
  - 422 -> request body hiba
- Válasz minta:

```
    {
        "id": 0,
        "question": "string",
        "correct_answer": "string",
        "quiz_id": 0,
        "explanation": "string",
        "created_at": "2019-08-24T14:15:22Z",
        "updated_at": "2019-08-24T14:15:22Z"
    }
```

3.2.9 Elérési utak: /api/courses/search/{name}/{category}

Az elérési út lekérdezi az összes kurzust, majd a név és a kategória alapján szűrteket visszaadja, a kategóriával együtt.

- Metódus: GET
- Válasz: a szűrési paramétereknek megfelelő kurzusok listája
- Státuszkódok: - 200 → sikeres
- Válasz minta:

```
[
    {
        "id": 2,
        "course_name": "Alias optio amet iusto qui voluptatem....",
        "created_at": "2026-04-10T14:10:07.000000Z",
        "updated_at": "2026-04-10T14:10:07.000000Z",
        "category_id": 3,
        "owner_id": 3,
        "max_level": 72882,
        "membercount": 84499,
        "price": 451181.179889,
        "primary_color": "#000000",
        "secondary_color": "#000000",
        "description": "Voluptatem....",
        "preview_video_url": "Quas quia laborum...",
```

```
        "course_logo": "Fuga labore magnam...",
        "banner_img": "Ipsum alias perferendis sit debitis...",
        "design_img": "Ut magnam sit eius omnis...",
        "side_img": "Voluptate architecto minima vedoloremque...",
        "pay_interval": "Fugit dolore.",
        "category": {
             "id": 3,
             "name": "Itaque et dolor voluptatem harum...",
             "created_at": "2026-04-10T14:10:07.000000Z",
             "updated_at": "2026-04-10T14:10:07.000000Z"
        }
   }
]
```


3.2.10 Hibakezelés

A HTTP request hibákat a Laravel beépített request-rendszere kezeli, ebből adódóan az összes hiba formátuma a következő:


```
   {
        "message": "string",
             "errors": {
             "property1": [
                  "string"
             ],
             "property2": [
                  "string"
             ]
        }
   }
```


3.2.11 Autentikáció

3.2.11.1 Védett utak

A legtöbb elérési utat Laravel api-token védi.
Tokent a fentebb tárgyalt /api/login és a /api/register szolgál ki.
Egy token formája a következő:
<szám>|<véletlenszerű karaktersor>
pl: 1|DJUV1df7rdcI3OXMMgbmPYWOOeN0lYaCjiFllfQGa444b793

3.2.12 Authorizáció

Bizonyos elérési utakat csak az adminisztrátor érhet el, ezt a User isadmin mezőjének ellenőrzésével biztosítjuk a releváns kontroller osztályok metódusaiban.

3.3 Modell osztályok

A Laravel Eloquent Model funkcióinak köszönhetően a modell osztály nem egy klasszikus modell osztály.
Hiszen az Eloquent Model amennyiben ismeri az adatbázis táblát, automatikusan felveszi annak mezőit.
Mivel a Laravel beépített users táblájának kiegészített verzióját használjuk, az UserModel speciális kialakítású.
Ezért csak egy szabványos modell, és a UserModel lesz részletesebben ismertetve.

3.3.1 Egy szabványos modell

```
class TransactionModel extends Model
{
    /**@useHasFactory<\Database\Factories\TransactionModelFactory>

    use HasFactory;

    protected $guarded = ["id"];

    protected $table = "transactions";
}
```

Az use HasFactory segítségével a modell tudja használni a hozzárendelt factory-t.
A Laravel elnevezés alapján automatikusan megkeresi a factory-t és használhatóvá teszi.
A $guarded mező segítségével a Mass-Assignment műveletektől védett mezőket lehet definiálni.
A $table pedig a modellhez rendelt adatbázis táblát tartalmazza.

3.3.2 UserModel.php

```
    /** @use HasFactory<\Database\Factories\UserModelFactory> */
    use HasFactory, Notifiable, HasApiTokens;

    protected $guarded = ["id"];

    protected $table = "users";

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var list<string>
     */
    protected $hidden = [
        'password',
        'remember_token',
    ];
```

```
    /**
     * Get the attributes that should be cast.
     *
     * @return array<string, string>
     */
    protected function casts(): array
    {
        return [
            'email_verified_at' => 'datetime',
            'password' => 'hashed',
        ];
    }
```

A szabványos modell funkcióit kiegészíti a casts metódus, ami automatikusan konvertálja a kiválasztott mezők típusát a return blokkban megadottra.

3.4 Adatbázis létrehozás és módosítás: migrációk

Bár az Adatbázis létrehozó SQL fájl elérhető a database/SQL/DBPGSqlDump.sql fájlban, az adatbázis létrehozásához a Laravel migráció implementációját alkalmaztuk, a következő okokból:
- Könnyebb hibakezelés
- Integráció a Laravel környezetbe és a verziókezelésbe
- Az adatbázis alaphelyzetbe állítható 1 paranccsal
- Automatikus adatgenerálás lehetősége

A migráció struktúra az SQL fájl felépítése alapján készült:
- 1-1 migráció a táblák létrehozására
- 1 migráció az indexek hozzáadására
- 1 migráció az idegen kulcsok létrehozására

Mivel a migrációk felépítése hasonló, ezért csak egy példát mutatunk be.

3.4.1 Táblalétrehozó migráció

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
```

```
        Schema::create('video_lessons',function (Blueprint $table)
        {
            $table->id();
            $table->integer("lesson_id",false,true);
            $table->string("video_url",512);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('video_lessons');
    }
};
```

A Laravel beépített Schema osztályából kiindulva, a $table változó különböző metódusaival létrehoztuk a mezőket.
Néhány ezekből, az összes megtalálható a https://laravel.com/docs/12.x/migrations#available-column-types oldalon
$table->id(): Létrehoz egy NOT NULL, AUTOINCREMENT, BIGINT mezőt id néven
$table->integer(<NAME>): Létrehoz egy szám mezőt NAME névvel
$table->string(<NAME>,<LENGTH>): Létrehoz egy LENGTH hosszú VARCHAR-t NAME néven

3.5 Adatbázis biztonságos elérése

Az adatbázis eléréshez, az ipari szabvány .env fájlt használtuk, ezzel rejtve el a titkos adatokat a nagyvilág elől.
A Laravel beépített $_ENV szuperglobális változóját kihasználva, globálisan használt, de kevésbé titkos adatokkal is kibővítettük az .env fájlt.
Ezek megtalálhatóak a .env.example fájlban.

3.6 Kontroller osztály

Hasonlóan a modell osztályokhoz, a Laravel kontroller osztályai sok beépített funkcióval rendelkeznek, ami rövidíti és sablonosítja a kódot.
Ezért csak egy szabványos kontrollert ismertetünk.

3.6.1 Egy szabványos kontroller

(A könnyebb olvasás érdekében csak a metódusok az osztályból)

```
    public function index()
    {
        return CourseModel::all();
    }
```

```
    public function store(StoreCourseModelRequest $request)
    {
        $data = CourseModel::create($request->validated());
        return $data;
    }

    public function show($id)
    {
        $Target = DB::table("courses")->where("id", "=", $id)
        ->get();
        if ($Target->isEmpty()) {
            return response(["message" => "Target not
             found"],404);
        }
        return $Target;
    }

    public function update(UpdateCourseModelRequest $request, $id)
    {
        DB::table("courses")->where("id", "=", $id)
        ->update($request->validated());
        $Updated = $this->show($id);
        return $Updated;

    }

    public function destroy($id)
    {
        $Result=DB::table("courses")>where("id","=",$id)→delete();
        if ($Result == 0) {
            return response(["message" => "Target for Deletion not
             found"],404);
        }
        return response()→noContent();
    }
```

Magyarázat:
- az index metódus visszaadja az összes adatot, a Model beépített statikus all() metódusával
- a store metódus a request-ből érkező, validált adatot menti el az adatbázisba, vagy 422-es státuszkóddal hibát dob
- a show metódus megkeresi a kért elemet id alapján, ha nem található, 404-es hibával tér vissza, ellenkező esetben a talált adattal
- az update metódus megkeresi a kért elemet, majd módosítja a validált request adatok alapján

- a destroy metódus megkeresi a kért elemet, majd törli ha létezik, vagy 404-es hibával tér vissza

3.7 Adatok validálása

Az adatokat a Laravel beépített request-ei validálják, így a kontrollerhez és a modellhez hasonlóan csak 1-et mutatunk be.
Külön request-eket hoztunk létre a POST és a PUT/PATCH HTTP metódusokhoz.

3.7.1 Egy szabványos request

(A könnyebb olvasás érdekében csak a metódusok az osztályból)

```
public function authorize(): bool    {
        return true;
    }

    public function rules(): array
    {
        return [
           "quiz_id"=>"required|integer|numeric|exists:quizes,id"
"user_course_profile_id"=>"required|integer|numeric|exists:user_co
urse_profiles,id",
            "correct_answer" => "required|string|max:4",
            "explanation" => "required|string|max:512"
        ];
    }
```

A rules metódusban találhatóak meg a mezők és szabályaik.
A szabályok teljes listája elérhető a https://laravel.com/docs/12.x/validation#available-validation-rules oldalon, csak néhány általunk gyakran használtat emelünk ki.
required: a mezőnek benne kell lennie a request body-ban
numeric: a mező(ha létezik) szám típusú
exists:<TABLE>,<CNAME>: a TABLE tábla CNAME mezőjében létezik a megadott érték
string: a mező(ha létezik) szöveg típusú
max:<AMNT>: a mező maximum AMNT karakter hosszú ha szöveg, vagy AMNT számjegyű ha szám

3.8 Minta adatok generálása

A minta adatokat a php artisan migrate:fresh --seed paranccsal lehet generálni.
A fenti parancs alaphelyzetbe állítja az adatbázist, ezért csak tesztelésnél érdemes használni.
Az adatmennyiség szabályozására van lehetőség, a .env-ben megtalálható FACTORY_ITEM_DB értékkel.
Egy tábla mintaadatokkal való feltöltésének folyamata a következő, és minden táblánál megegyezik, ezért ismételten csak egy kerül bemutatásra.
Factory létrehozása -> Seederben meghívás -> parancs futtatása

3.8.1 Egy szabványos factory

(A könnyebb olvasás érdekében csak a metódusok az osztályból)

```
    public function definition(): array
    {
        return [
             "course_id"=>fake()
             ->numberBetween(1,$_ENV["FACTORY_ITEM_DB"]),
            "format_type" => fake()→text(16),
            "resource_url" => fake()->text(512),
        ];
    }
```

A szintaxis a migrációk szintaxisára hasonlít, az összes metódusa a fake()-nek elérhető a https://fakerphp.org dokumentációjában.
Néhányat ennek ellenére kiemelünk:
text(<LEN>): LEN hosszúságú szöveg
numberBetween(<MIN>,<MAX>): MIN és MAX közötti szám
boolean(): logikai érték

3.8.2 DatabaseSeeder.php

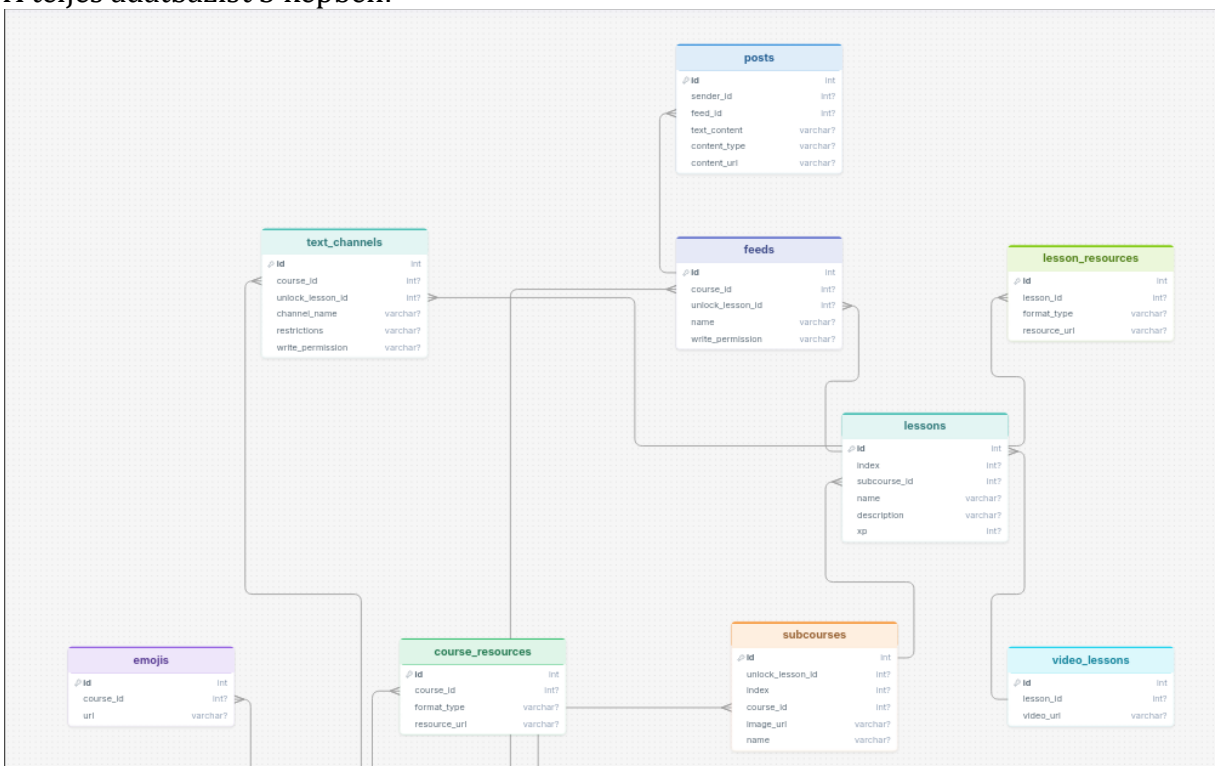
A seeder run metódusa a következő sorokat tartalmazza
CourseResourceModel::factory($_ENV["FACTORY_ITEM_DB"])->create();
A parancs futtatásakor sorban lefutnak a factory-k.
A sorrend fontos, az idegen kulcsok miatt.

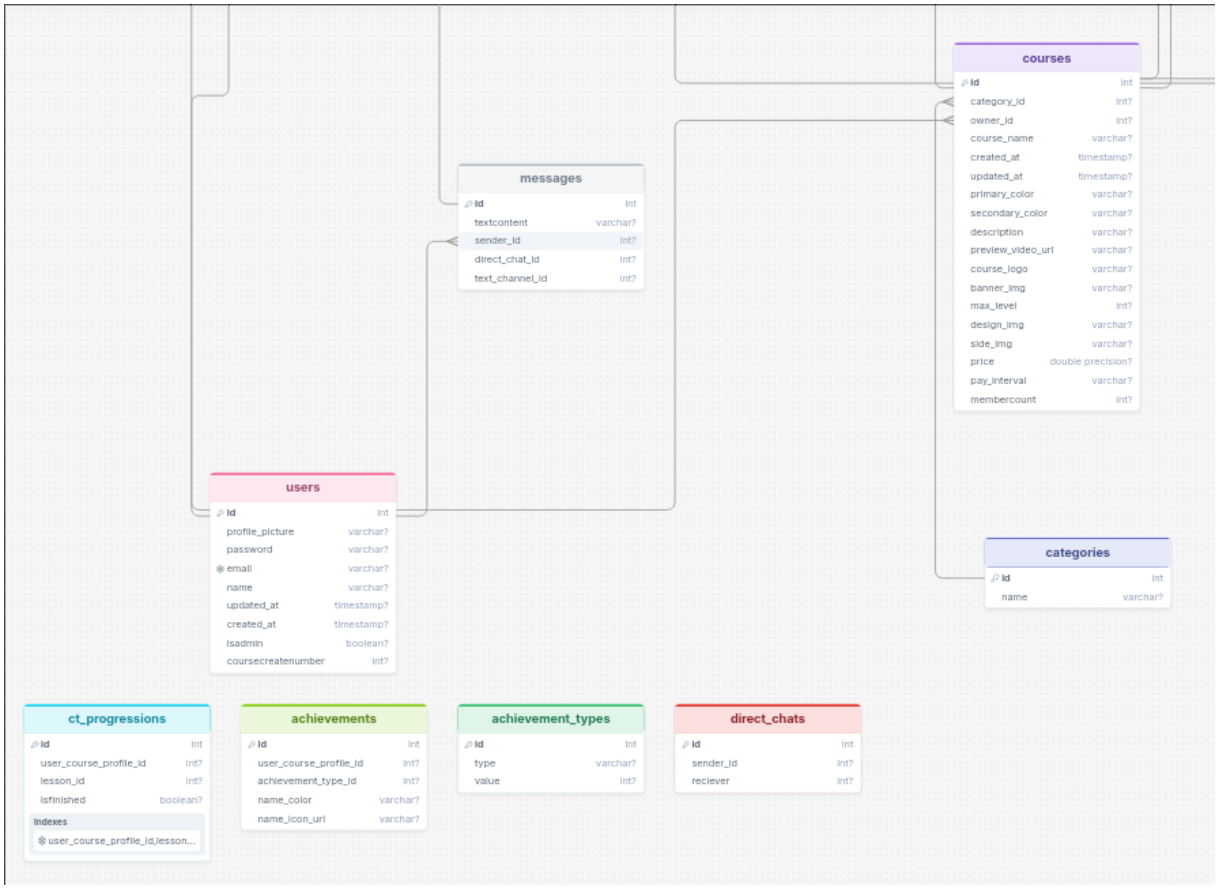
3.9 Adatbázis

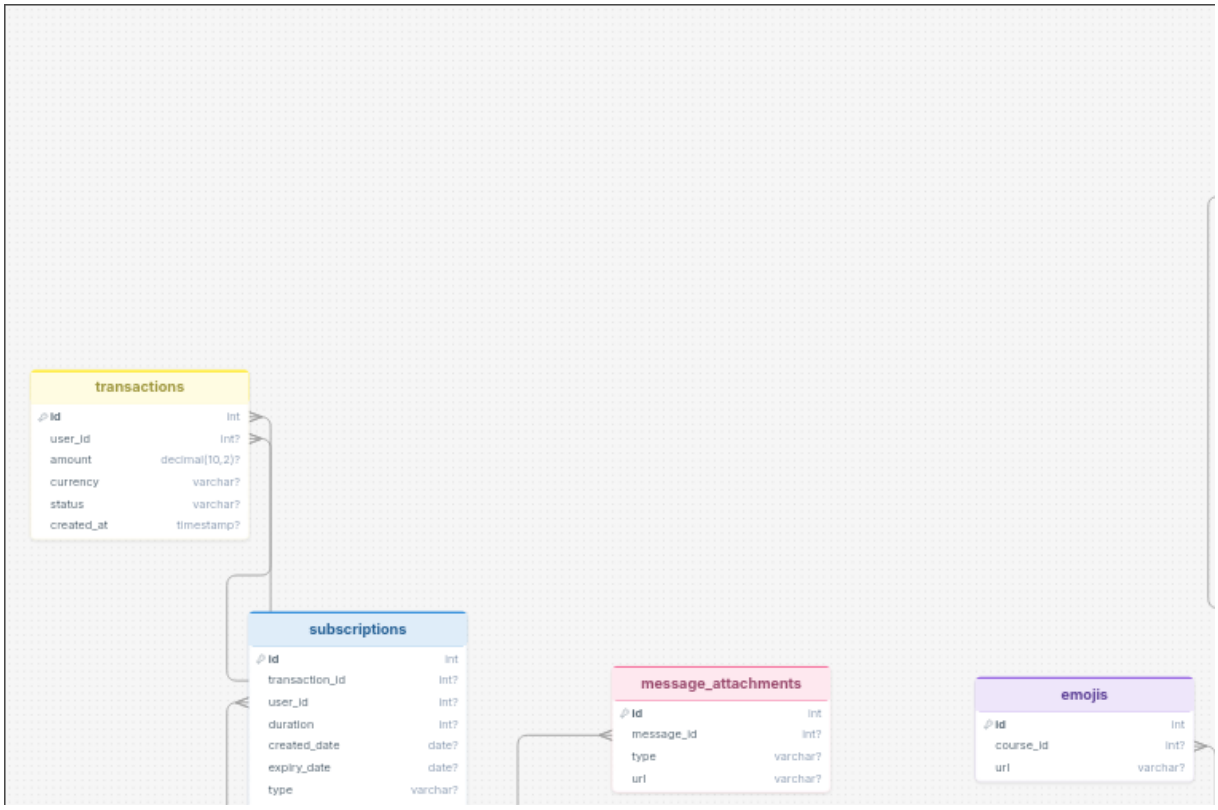
Az adatperzisztencia és az adatok megbízható tárolása céljából a platform mögött egy PostgreSQL relációs adatbázis-kezelő rendszert alkalmaztunk. A választásunk azért esett erre a robusztus, nyílt forráskódú technológiára, mert a Skilltree adatmodellje – a felhasználók, kurzusok, leckék, jogosultságok és chat üzenetek hálózata – rendkívül sokrétű, egymással szorosan összefüggő relációkból áll.

3.9.1 Adatbázis táblái

Az adatbázis vizualizálására/megtervezésére DBDiagramot használtunk.
A teljes adatbázist 3 képben:




courses
id int
category_id int?
owner_id int?
course_name varchar?
created_at timestamp?
updated_at timestamp?
primary_color varchar?
secondary_color varchar?
description varchar?
preview_video_url varchar?
course_logo varchar?
banner_img varchar?
max_level int?
design_img varchar?
side_img varchar?
price double precision?
pay_interval varchar?
membercount int?
messages
id int
textcontent varchar?
sender_id int?
direct_chat_id int?
text_channel_id int?
users
id int
profile_picture varchar?
password varchar?
email varchar?
name varchar?
updated_at timestamp?
created_at timestamp?
isadmin boolean?
coursecreatenumber int?
categories
id int
name varchar?
ct_progressions
id int
user_course_profile_id int?
lesson_id int?
isfinished boolean?
indexes
user_course_profile_id,lesson...
achievements
id int
user_course_profile_id int?
achievement_type_id int?
name_color varchar?
name_icon_url varchar?
achievement_types
id int
type varchar?
value int?
direct_chats
id int
sender_id int?
reciever int?


transactions
id int
user_id int?
amount decimal(10,2)?
currency varchar?
status varchar?
created_at timestamp?
subscriptions
id int
transaction_id int?
user_id int?
duration int?
created_date date?
expiry_date date?
type varchar?
message_attachments
id int
message_id int?
type varchar?
url varchar?
emojis
id int
course_id int?
url varchar?

4 Frontend

4.1 Bevezetés

A projekt keretében kifejlesztett Next.js frontend alkalmazás egy modern, reszponzív webes felületet biztosít az oktatási és közösségi funkciók zökkenőmentes eléréséhez. A Next.js App Router architektúrájára és Tailwind CSS keretrendszerre épülő alkalmazás két fő részből áll: a nyilvános látogatói felületből és egy hitelesítést igénylő privát zónából (amely a kurzusokat és a valós idejű chatet is magába foglalja), melyeket egy biztonságos munkamenet-kezelő rendszer köt össze.

4.2 Frontend Architektúra és Kliens-Szerver Kapcsolat

A projekt felhasználói felületének alapját a Next.js keretrendszer adja. A tervezés során nagy hangsúlyt fektettünk arra, hogy a modern webfejlesztési irányelveket követve egy olyan architektúrát alakítsunk ki, amely egyszerre biztosít gyors betöltési időt, biztonságos adatkezelést és kiváló felhasználói élményt. Ennek a megközelítésnek a magja a szerver és a böngésző (kliens) feladatainak éles szétválasztása.

4.2.1 A Next.js App Router működési elve

A frontend felépítéséhez a Next.js legújabb, App Router nevű architektúráját választottuk. Eltérően a hagyományos React alkalmazásoktól, itt az útvonalválasztás (routing) közvetlenül a fájlrendszerre épül. Az app mappán belüli mappaszerkezet határozza meg az alkalmazás URL címeit, ami átláthatóbbá tette a kódbázist, és lehetővé tette a Layoutok (például navigációs sávok) logikus egymásba ágyazását.

4.2.2 Szerver Komponensek (Server Components) a gyakorlatban

A Next.js App Router alapértelmezetten minden komponenst a szerveren renderel ki. Ezek a komponensek nem küldenek JavaScriptet a felhasználó böngészőjébe, csupán a felépített HTML kódot, ami drasztikusan javítja a teljesítményt.
Rendszerünkben a nehezebb számítási feladatokat és az aszinkron adatlekéréseket (például a felhasználói munkamenet ellenőrzését) a szerverre bíztuk. Kiváló példa erre a CourseCard és a DiscordChatSection komponensünk (ez a komponens jeleníti meg egy oldalon, egy adott chat-szobában az elküldött üzeneteket). Ahogy a lenti kódrészleten is látszik, a komponens aszinkron módon, a háttérben kéri le a munkamenetet (GetSession()), így a klienshez már csak a biztonságos, kész adatok érkeznek meg.

DiscordChatSection.tsx (Részlet) - Szerver Komponens

```
export default async function DiscordChatSection({
  courseName,
  memberCount,
  primaryColor,
  channels,
  members,
```

```
}: DiscordChatSectionProps) {
  let isSetChannel: boolean = false;
  // Szerveroldali adatlekérés és jogosultság-ellenőrzés
  const session = await GetSession();

  if (!session.activetextchannelid) {
       isSetChannel = false;
  } else {
       isSetChannel = true;
  }

  return (
    <section className="px-4 pb-20 pt-6 sm:px-6 lg:px-8">
      {/* ... Vizuális elemek ... */}
      <MessagesSection
activetextchannelid={session.activetextchannelid} />
      <InputMessage                    primaryColor={primaryColor}
isSelectedChannel={isSetChannel} />
    </section>
  );
}
```

4.2.3 Kliens Komponensek (Client Components) és Interaktivitás

Azokat a fájlokat, ahol interaktivitásra vagy folyamatos állapotfrissítésre volt szükség, a "use client" direktívával láttuk el. Ezeket kizárólag a dinamikus felületeknél alkalmaztuk.

Tökéletes példa erre a MessagesSection és a MessageItem komponensünk. A MessagesSection felel a chat üzenetek folyamatos (polling) frissítéséért. Itt a React beépített állapotkezelőit (useState, useEffect) használtuk, amelyek csak a kliensoldalon futnak le.

MessagesSection.tsx (Részlet) - Kliens Komponens

```
"use client";

import React, { useEffect, useState } from 'react';
import MessageItem from './MessageItem'

export  default  function  MessagesSection({  activetextchannelid,
pollIntervalMs = 4000 }) {
  // Állapotkezelés a kliensen
  const [messages, setMessages] = useState<MessageWithUser[]>([]);

  useEffect(() => {
    let cancelled = false;

    // Folyamatos adatlekérés (polling) a szervertől a háttérben
```

```
    const tick = async () => {
      if (!activetextchannelid) return;
      const res = await
fetch(`/api/text_channels/${activetextchannelid}/messages/0`, {
        method: "GET",
        headers: { accept: "application/json" },
      });
      // ... adatok feldolgozása ...
      setMessages(next);
    };

    const intervalId = window.setInterval(tick, pollIntervalMs);
    return () => window.clearInterval(intervalId);
  }, [activetextchannelid, pollIntervalMs]);

  return (
    <div className="min-h-0 flex-1 space-y-4 overflow-y-auto p-4
sm:p-5">
      {messages.map((message) => (
        <MessageItem key={message.id} message={message} />
      ))}
    </div>
  );
}
```

4.2.4 Adatfolyam és kommunikáció a két réteg között

A rendszer adatintegritását a komponensek közötti egyirányú adatfolyam biztosítja. A fenti két kód kiválóan szemlélteti ezt a kapcsolatot: a szerveroldali DiscordChatSection lekéri a biztonságos adatokat (például a session.activetextchannelid értékét), majd ezt az adatot paraméterként (prop-ként) adja át a kliensoldali MessagesSection komponensnek.
Ezzel a megközelítéssel elértük, hogy a böngésző csak a megjelenítésért és az interakciókért feleljen, miközben a biztonságos üzleti logika elrejtve, a backendhez közel, a Next.js szerveren fut.

4.3 Adatlekérés (Data Fetching) és API Kommunikáció

Egy modern webalkalmazás sebességét és felhasználói élményét alapvetően meghatározza az, hogy milyen stratégiát alkalmazunk az adatok betöltésére. A Next.js App Router ebben is új paradigmát hozott: lehetővé tette számunkra, hogy az adatlekéréseket a lehető legközelebb vigyük a backendhez, miközben a kliensoldali dinamizmust is megőrizzük. Rendszerünkben egy hibrid megközelítést alkalmaztunk, kihasználva a szerveroldali és a kliensoldali adatlekérések előnyeit is.

4.3.1 Szerveroldali adatlekérési stratégiák

A kezdeti adatbetöltéseket – ahol csak lehetséges volt – a Szerver Komponensekbe mozgattuk. Mivel ezek a komponensek a szerveren futnak le, a natív fetch API-t,

illetve a közvetlen aszinkron függvényeket (például a GetSession()) közvetlenül a komponens törzsében tudtuk meghívni.
Ennek a stratégiának számos előnye volt a projektünk szempontjából:

- **Biztonság:** Az érzékeny végpontok, API kulcsok vagy az adatbázis-kapcsolatok részletei soha nem kerülnek ki a kliensoldalra.
- **Teljesítmény:** Az adatlekérés és a HTML generálás ugyanazon a szerveren történik, így megspóroljuk a hálózati késleltetést (latency), ami a kliensről indított kéréseknél jelentkezne.
- **Közvetlen hozzáférés a munkamenethez:** Ahogy a kurzusok megjelenítésénél is tettük, a szerveroldalon azonnal ellenőrizni tudtuk a felhasználó jogosultságait, mielőtt az oldal egyáltalán betöltődött volna a böngészőben.

4.3.2 Aszinkron kérések és adatbetöltés kezelése a felhasználói felületen

Bizonyos funkciók esetében a szerveroldali adatlekérés nem elegendő, hiszen az adatoknak a felhasználói interakciók vagy az idő múlásának hatására (újratöltés nélkül) kell frissülniük. Ezeket a feladatokat a Kliens Komponensekben kezeltük.
A kliensoldali API kommunikáció során a következő elveket követtük:

- **Dinamikus adatfrissítés (Polling):** A valós idejűnek szánt felületeken (mint például a MessagesSection chat modulunk) beépített React hook-okat (useEffect) használtunk a háttérben futó, ismétlődő adatlekérésekre. Ez biztosítja, hogy a felhasználók folyamatosan lássák a beérkező új üzeneteket.
- **Hibakezelés és Útválasztás:** Minden kliensoldali API hívásnál felkészültünk az esetleges hálózati hibákra vagy a lejárt munkamenetekre. Például, ha a végpont 401-es (Unauthorized) hibakódot ad vissza, a rendszer automatikusan átirányítja a felhasználót a bejelentkezési oldalra (window.location.href = "/login").
- **Állapotkezelés (State Management):** A beérkező nyers adatokat (payload) saját típusdefinícióinknak (például MessageWithUser) megfelelően validáltuk és dolgoztuk fel, majd a React useState hook-jának segítségével frissítettük velük a felületet.

Ez a kettős felosztás biztosította, hogy az alkalmazásunk gyorsan töltődjön be, SEO-barát maradjon, de a közösségi és interaktív funkciók is zökkenőmentesen működjenek.
A kliensoldali adatlekérés és hibakezelés gyakorlati megvalósítását jól szemlélteti a beépített chat modulunk (MessagesSection) kódrészlete. Az alábbi példában egy aszinkron API hívást hajtunk végre, amely a folyamatos frissítés (polling) részeként kérdezi le az új üzeneteket a szervertől:

```
const res = await
fetch(`/api/text_channels/${activetextchannelid}/messages/0`, {
  method: "GET",
  headers: {
    accept: "application/json",
  },
  cache: "no-store",
  credentials: "include",
  signal: controller.signal,
```

```
});

if (res.status === 401) {
  window.location.href = "/login";
  return;
}
```

A fenti implementációban nemcsak az adatok valós idejű, gyorsítótárazás nélküli (cache: "no-store") lekérését biztosítottuk, hanem a robusztus hibakezelést is. Ha az API végpont 401-es (Unauthorized) hibakóddal tér vissza – például mert időközben lejárt a munkamenet –, a kliens azonnal megszakítja a folyamatot, és a bejelentkezési oldalra irányítja a felhasználót, védve ezzel az alkalmazás biztonságát.

4.4 Munkamenet-kezelés és Hitelesítés

Az alkalmazás biztonságának és a személyre szabott felhasználói élménynek az alapja a munkamenet-kezelés (session management). A rendszerünkben a hitelesítés feladata, hogy kaput képezzen a nyilvános látogatói felület és a privát funkciók – például a kurzusokon való részvétel vagy a közösségi chat – között.

4.4.1 Szerveroldali munkamenet-ellenőrzés

A Next.js App Router adta lehetőségeket kihasználva a hitelesítést elsősorban a szerveroldalon kezeljük. Erre a célra egy központosított GetSession() aszinkron függvényt alkalmazunk, amely közvetlenül a Szerver Komponensekben kérdezi le az aktuális felhasználó adatait és jogosultságait.
Ez a megközelítés garantálja, hogy:

- A privát adatok (például a felhasználó e-mail címe vagy egyedi azonosítója) ne szivárogjanak ki a kliensoldalra.
- A rendszer már a HTML generálása előtt dönteni tudjon arról, hogy a felhasználó jogosult-e az adott tartalom megtekintésére.

4.4.2 Feltételes renderelés és hozzáférés-kezelés

A munkamenet állapotát nemcsak az oldalak védelmére, hanem a komponensek szintjén is felhasználjuk. A felhasználói felület dinamikusan alkalmazkodik a bejelentkezett állapothoz: például a kurzuskártyákon csak hitelesített munkamenet esetén jelenik meg a „Join” gomb, ellenkező esetben a rendszer korlátozza az interakciót.
Példa a munkamenet-alapú feltételes renderelésre

```
const session = await GetSession();

return (
  <div className="mt-auto flex items-center justify-end">
    {session.user != null ? <JoinButton
courseId={Number(course.id)} />  : ""}
  </div>
);
```

4.4.3 Kliensoldali védelem és munkamenet-lejárat

Bár a hitelesítés központja a szerver, a Kliens Komponenseknek is tudatosan kell kezelniük a munkamenetet. Amennyiben egy felhasználó hosszabb ideig inaktív marad, és a munkamenete lejár, a kliensoldali API hívások (mint például az üzenetek lekérése) HTTP 401-es hibát kapnak. Ebben az esetben az alkalmazás automatikusan megszakítja az aktuális folyamatot, és biztonsági okokból visszairányítja a felhasználót a bejelentkezési felületre.
Ez a hibrid biztonsági architektúra biztosítja, hogy a Skilltree rendszere egyszerre legyen kényelmes a felhasználók számára és ellenálló a jogosulatlan hozzáférésekkel szemben.

4.5 Felhasználói felület és komponenskönyvtár

A Skilltree platform vizuális megjelenésének kialakításakor a fő cél egy modern, letisztult, sötét témájú (dark mode) felület megalkotása volt, amely a gamerek és az e-learning rendszerek felhasználói számára egyaránt ismerős és kényelmes. Az egyedi CSS fájlok írása helyett a **Tailwind CSS** keretrendszert alkalmaztuk, amely "utility-first" (eszközosztály-alapú) megközelítésével tette lehetővé a gyors és konzisztens UI fejlesztést.

4.5.1 Reszponzivitás és elrendezés (Layout)

Az alkalmazást mobil-első (mobile-first) szemlélettel terveztük, így a felület minden eszközön optimálisan jelenik meg. A komplexebb nézeteknél – mint amilyen a közösségi chat felület (Community Hub) – a CSS Grid és Flexbox elrendezéseket használtuk a struktúra kialakítására.
Ahogy a kódstruktúrában is látható, a Tailwind reszponzív prefixeivel (sm:, lg:) oldottuk meg a képernyőmérettől függő alkalmazkodást. Például a chat felület asztali nézetben egy háromoszlopos rácsot (lg:grid-cols-[240px_minmax(0,1fr)_220px]) használ, míg kisebb képernyőkön a szélső oszlopok rejtve maradnak vagy összecsukódnak.

4.5.2 Dinamikus stílusozás és arculat

A Skilltree egyik különleges vizuális funkciója, hogy a kurzusok és csatornák megjelenése alkalmazkodik az adatbázisból érkező egyedi színekhez (elsődleges és másodlagos színkódok). Ezt a React beépített stíluskezelésének és a Tailwind CSS-nek az ötvözésével értük el.
A komponensek inline stílusként (CSS változókként) kapják meg a szerverről érkező biztonságos színkódokat, amelyeket aztán a felület különböző pontjain (pl. gombok háttere, színátmenetes fejlécek) hasznosítanak:
Példa a dinamikus színkezelésre a Skilltree felületén

```
<div
  className="relative aspect-video w-full"
  style={{ background: `linear-gradient(120deg, ${safePrimary}, ${safeSecondary})` }}
>
  {/* A kurzus egyedi színátmenetes fejléce */}
```

```
</div>
```

4.5.3 Interaktív vizuális visszajelzések és mikro-animációk

A magas szintű felhasználói élmény (UX) érdekében kiemelt figyelmet fordítottunk az interakciókra. A komponensek – például a kurzuskártyák (CourseCard) – hover (rámutatás) állapotában finom CSS animációkat alkalmaztunk. A transition, duration-300, és hover:scale osztályok használatával olyan modern vizuális elemeket hoztunk létre, mint az áttetsző üveghatás (backdrop-blur-sm) és az interaktív árnyékolás, amelyek élőbbé és professzionálisabbá teszik a Skilltree felületét.

4.6 Funkcionális modulok: Kurzuskezelés és Tartalomkezelés

A Skilltree platform központi eleme a tudásmegosztás, amelyet két fő funkcionális egység támogat: a kurzusok böngészése és a komplex kurzusépítő modul. Ezek a felületek szoros egységben működnek, biztosítva, hogy a tartalomkészítés és a tanulás folyamata is gördülékeny legyen.

4.6.1 Kurzusmegjelenítés és böngészés

A felhasználók a hitelesítés után egy dinamikus kurzuslistával találkoznak. Itt a korábban bemutatott CourseCard komponensek segítségével, kategóriákra bontva láthatják az elérhető képzéseket. Minden kártya vizuálisan tükrözi a kurzus egyedi arculatát (színátmenetek), és azonnali információt ad a kurzus szintjéről, a tagok számáról és a tartalomról. A frontend a szerveroldali adatok alapján dönti el, hogy a felhasználó már tagja-e az adott kurzusnak, vagy "Join" művelettel csatlakozhat.

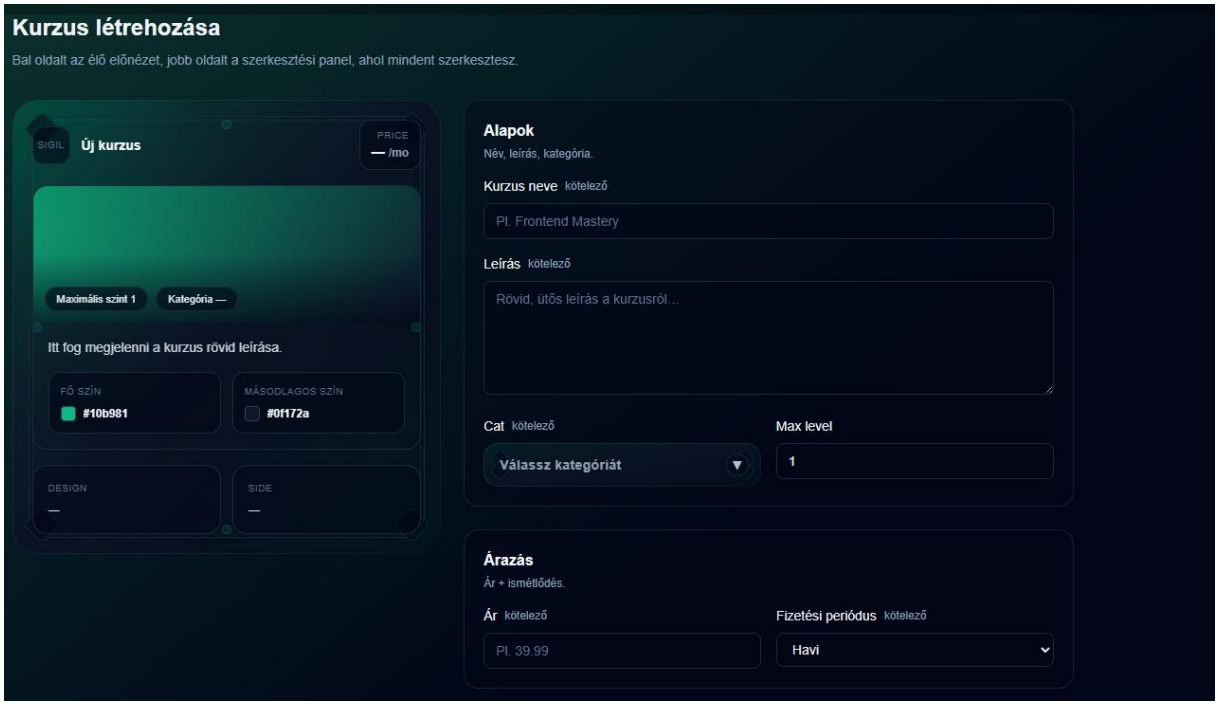
4.6.2 A Kurzus építő (Course Builder) architektúrája

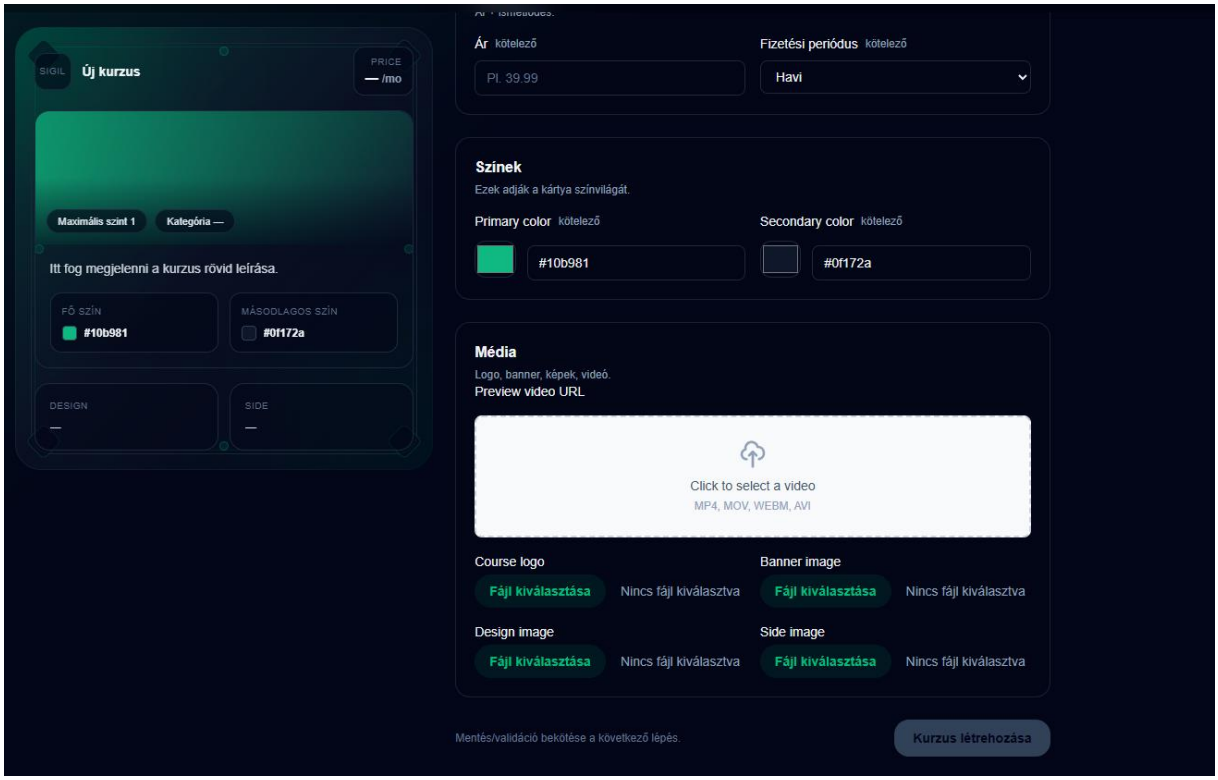
A Skilltree egyik legösszetettebb része a kurzusépítő felület, amely lehetővé teszi a tartalomkészítők számára, hogy hierarchikus struktúrába rendezzék a tudásanyagot. A tervezés során a modularitásra törekedtünk, így a felépítés három szinten valósul meg:

1. **Kurzus (Course):** A legfelsőbb szint, amely az általános leírást, arculatot és jogosultságokat tartalmazza.
2. **Modulok és Alkurzusok (Subcourses):** Logikai egységek, amelyek a kurzust nagyobb fejezetekre bontják.
3. **Leckék (Lessons):** A tényleges tananyag (szöveg, videó vagy egyéb média) alapkövei.

Az építő modul kliensoldali interaktivitásra épül (React állapotkezelés), ahol a készítő valós időben láthatja a kurzus struktúrájának alakulását. A frontend dinamikusan kezeli az új elemek hozzáadását, és azonnal validálja a bemeneti adatokat, mielőtt azok az API-n keresztül véglegesítésre kerülnének az adatbázisban.

A kurzusépítő felülete a weboldalon(két képben):


Kurzus létrehozása
Bal oldalt az élő előnézet, jobb oldalt a szerkesztési panel, ahol mindent szerkesztesz.
SIGIL
Új kurzus
PRICE
— /mo
Maximális szint 1
Kategória —
Itt fog megjelenni a kurzus rövid leírása.
FŐ SZÍN
#10b981
MÁSODLAGOS SZÍN
#0f172a
DESIGN
—
SIDE
—
Alapok
Név, leírás, kategória.
Kurzus neve kötelező
Pl. Frontend Mastery
Leírás kötelező
Rövid, ütős leírás a kurzusról…
Cat kötelező
Válassz kategóriát
Max level
1
Árazás
Ár + ismétlődés.
Ár kötelező
Pl. 39.99
Fizetési periódus kötelező
Havi


SIGIL
Új kurzus
PRICE
— /mo
Maximális szint 1
Kategória —
Itt fog megjelenni a kurzus rövid leírása.
FŐ SZÍN
#10b981
MÁSODLAGOS SZÍN
#0f172a
DESIGN
—
SIDE
—
Ár kötelező
Pl. 39.99
Fizetési periódus kötelező
Havi
Színek
Ezek adják a kártya színvilágát.
Primary color kötelező
#10b981
Secondary color kötelező
#0f172a
Média
Logo, banner, képek, videó.
Preview video URL
Click to select a video
MP4, MOV, WEBM, AVI
Course logo
Fájl kiválasztása
Nincs fájl kiválasztva
Banner image
Fájl kiválasztása
Nincs fájl kiválasztva
Design image
Fájl kiválasztása
Nincs fájl kiválasztva
Side image
Fájl kiválasztása
Nincs fájl kiválasztva
Mentés/validáció bekötése a következő lépés.
Kurzus létrehozása

4.7 Adminisztrációs felület és Jogosultságkezelés

A Skilltree platform zavartalan működésének és moderálásának elengedhetetlen feltétele egy dedikált adminisztrációs felület. Ezen a biztonságos, elzárt területen a rendszergazdák közvetlen adatbázis-hozzáférés nélkül, egy grafikus felületen keresztül tudják menedzselni a felhasználókat, a kurzusokat és a platform egyéb beállításait.

4.7.1 Szerepkör-alapú hozzáférés-vezérlés (RBAC)

Az adminisztrációs felület védelme az alkalmazás legmagasabb prioritású biztonsági feladata. Ennek megvalósítására egy szerepkör-alapú hozzáférés-vezérlési (Role-Based Access Control) rendszert alakítottunk ki.
A Next.js App Router Layout rendszerét kihasználva az adminisztrációs útvonalakat (/admin/*) egy közös, szerveroldali védőréteggel láttuk el. Amikor egy lekérés érkezik ezekre a végpontokra, a rendszer a GetSession() segítségével először megvizsgálja a felhasználó munkamenetét. Amennyiben a felhasználó nem rendelkezik a megfelelő (például ADMIN) jogosultsággal, a szerver a felület kirenderelése nélkül, azonnal átirányítja őt a főoldalra. Ezzel a módszerrel a jogosulatlan felhasználók még az adminisztrációs komponensek JavaScript kódjához sem férhetnek hozzá.

4.7.2 Rendszerfelügyelet és Adatkezelés (CRUD)

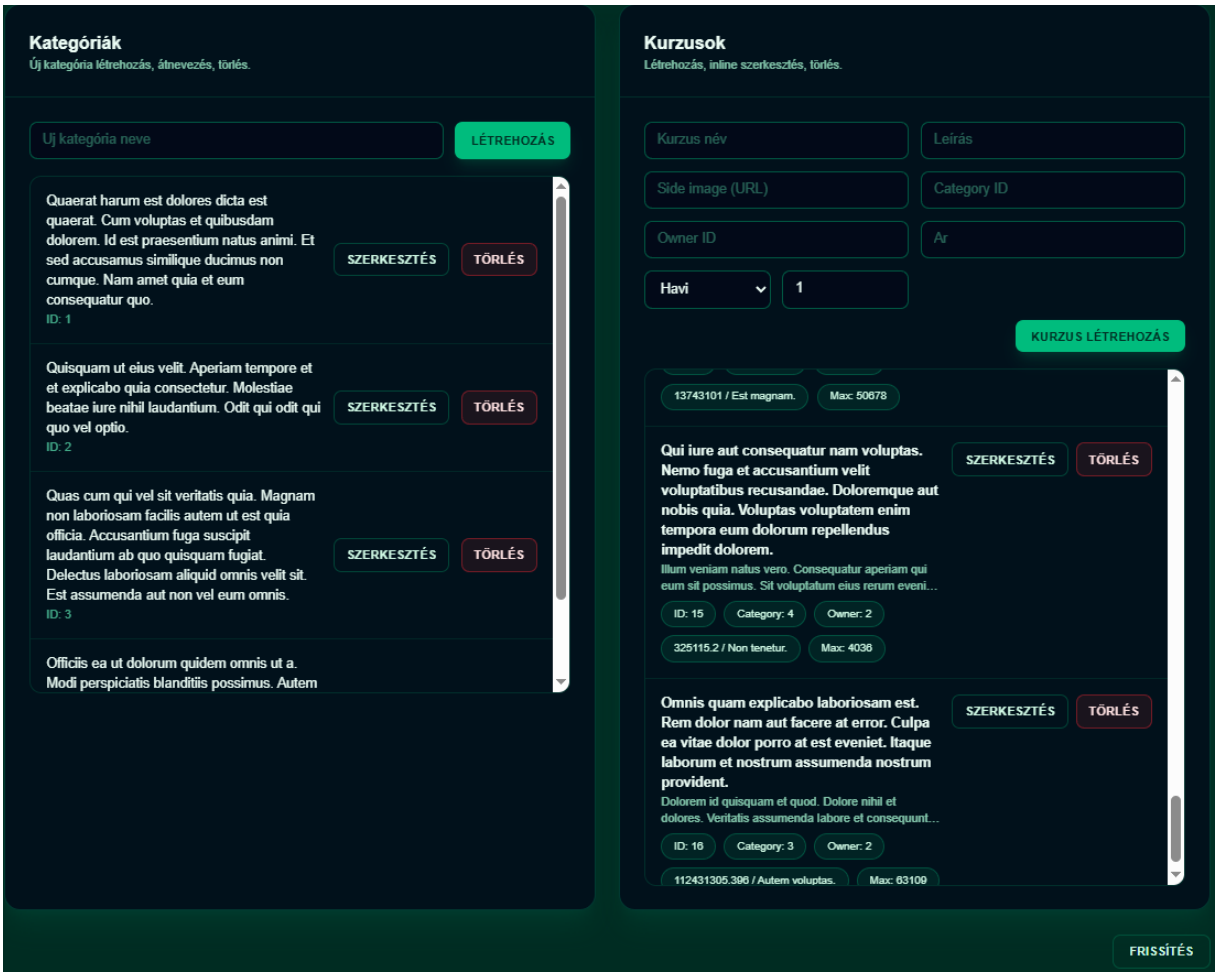
Az adminisztrációs modul fő funkciója az adatok teljes körű kezelése (Create, Read, Update, Delete - CRUD). A felület több logikai egységre bomlik:

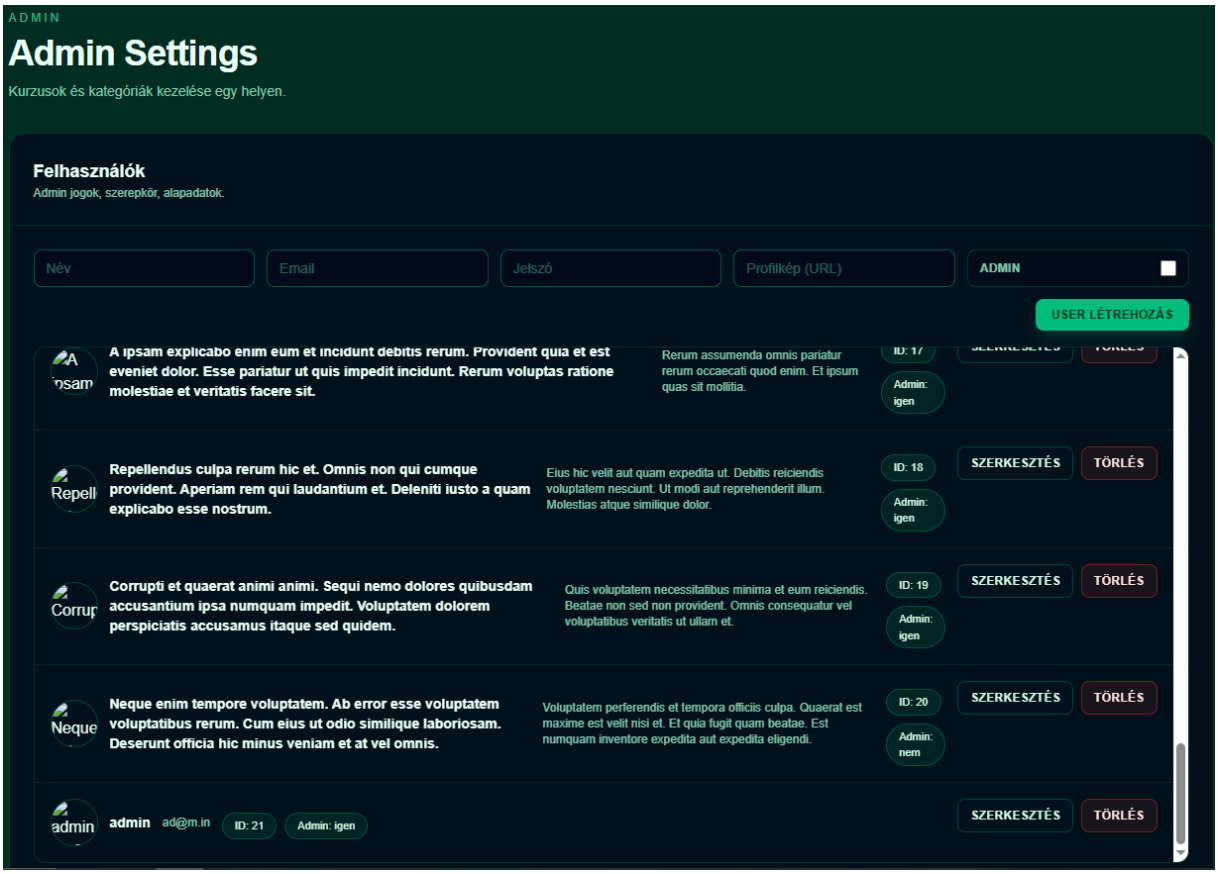
- **Felhasználókezelés:** Az adminisztrátorok áttekinthetik a regisztrált tagok listáját, módosíthatják a jogosultságaikat (például oktatói státusz adományozása), vagy szükség esetén felfüggeszthetik a fiókjaikat.
- **Kurzus- és Kategóriakezelés:** A platformon megjelenő tartalmak felügyelete, új kategóriák létrehozása, valamint a nem megfelelő tartalmak eltávolítása.
- **Kommunikációs csatornák (Community Hub) moderálása:** A platformhoz tartozó szöveges csatornák és a hozzájuk tartozó beállítások kezelése.

Ezeknél a funkcióknál a kliensoldali űrlapok szorosan együttműködnek a védett backend API végpontokkal, minden egyes adatmódosítási kérésnél újra validálva a felhasználói jogosultságokat.

4.7.3 Az adminisztrációs felhasználói felület ergonómiája

Vizuális szempontból az adminisztrációs felület eltér a publikus oldalak tartalomközpontú dizájnjától, és inkább az adatsűrűségre, valamint a funkcionalitásra fókuszál. A Tailwind CSS segítségével átlátható adattáblázatokat (Data Tables), rögzített oldalsó navigációs sávot (Sidebar) és felugró ablakokat (Modals) alakítottunk ki a megerősítést igénylő műveletekhez (például egy kurzus törléséhez). Ez az elrendezés biztosítja, hogy a rendszergazdák hatékonyan, nagy mennyiségű adatot is átlássanak anélkül, hogy a platform modern, sötét témájú vizuális identitása csorbát szenvedne.
Képek az admin felületről:


Kategóriák
Új kategória létrehozás, átnevezés, törlés.
Uj kategória neve
LÉTREHOZÁS
Quaerat harum est dolores dicta est quaerat. Cum voluptas et quibusdam dolorem. Id est praesentium natus animi. Et sed accusamus similique ducimus non cumque. Nam amet quia et eum consequatur quo.
ID: 1
SZERKESZTÉS
TÖRLÉS
Quisquam ut eius velit. Aperiam tempore et et explicabo quia consectetur. Molestiae beatae iure nihil laudantium. Odit qui odit qui quo vel optio.
ID: 2
SZERKESZTÉS
TÖRLÉS
Quas cum qui vel sit veritatis quia. Magnam non laboriosam facilis autem ut est quia officia. Accusantium fuga suscipit laudantium ab quo quisquam fugiat. Delectus laboriosam aliquid omnis velit sit. Est assumenda aut non vel eum omnis.
ID: 3
SZERKESZTÉS
TÖRLÉS
Officiis ea ut dolorum quidem omnis ut a. Modi perspiciatis blanditiis possimus. Autem
Kurzusok
Létrehozás, inline szerkesztés, törlés.
Kurzus név
Leírás
Side image (URL)
Category ID
Owner ID
Ar
Havi
1
KURZUS LÉTREHOZÁS
13743101 / Est magnam.
Max: 50678
Qui iure aut consequatur nam voluptas. Nemo fuga et accusantium velit voluptatibus recusandae. Doloremque aut nobis quia. Voluptas voluptatem enim tempora eum dolorum repellendus impedit dolorem.
Illum veniam natus vero. Consequatur aperiam qui eum sit possimus. Sit voluptatum eius rerum eveni...
ID: 15
Category: 4
Owner: 2
325115.2 / Non tenetur.
Max: 4036
SZERKESZTÉS
TÖRLÉS
Omnis quam explicabo laboriosam est. Rem dolor nam aut facere at error. Culpa ea vitae dolor porro at est eveniet. Itaque laborum et nostrum assumenda nostrum provident.
Dolorem id quisquam et quod. Dolore nihil et dolores. Veritatis assumenda labore et consequunt...
ID: 16
Category: 3
Owner: 2
112431305.396 / Autem voluptas.
Max: 63109
SZERKESZTÉS
TÖRLÉS
FRISSÍTÉS


ADMIN
Admin Settings
Kurzusok és kategóriák kezelése egy helyen.
Felhasználók
Admin jogok, szerepkör, alapadatok.
Név
Email
Jelszó
Profilkép (URL)
ADMIN
USER LÉTREHOZÁS
A ipsam explicabo enim eum et incidunt debitis rerum. Provident quia et est eveniet dolor. Esse pariatur ut quis impedit incidunt. Rerum voluptas ratione molestiae et veritatis facere sit.
Rerum assumenda omnis pariatur rerum occaecati quod enim. Et ipsum quas sit mollitia.
ID: 17
Admin: igen
Repellendus culpa rerum hic et. Omnis non qui cumque provident. Aperiam rem qui laudantium et. Deleniti iusto a quam explicabo esse nostrum.
Eius hic velit aut quam expedita ut. Debitis reiciendis voluptatem nesciunt. Ut modi aut reprehenderit illum. Molestias atque similique dolor.
ID: 18
Admin: igen
SZERKESZTÉS
TÖRLÉS
Corrupti et quaerat animi animi. Sequi nemo dolores quibusdam accusantium ipsa numquam impedit. Voluptatem dolorem perspiciatis accusamus itaque sed quidem.
Quis voluptatem necessitatibus minima et eum reiciendis. Beatae non sed non provident. Omnis consequatur vel voluptatibus veritatis ut ullam et.
ID: 19
Admin: igen
SZERKESZTÉS
TÖRLÉS
Neque enim tempore voluptatem. Ab error esse voluptatem voluptatibus rerum. Cum eius ut odio similique laboriosam. Deserunt officia hic minus veniam et at vel omnis.
Voluptatem perferendis et tempora officiis culpa. Quaerat est maxime est velit nisi et. Et quia fugit quam beatae. Est numquam inventore expedita aut expedita eligendi.
ID: 20
Admin: nem
SZERKESZTÉS
TÖRLÉS
admin
admin
ad@m.in
ID: 21
Admin: igen
SZERKESZTÉS
TÖRLÉS

4.8 Közösségi tér és Valós idejű kommunikáció (Community Hub)

A Skilltree platform nem csupán egy egyirányú oktatási felület, hanem egy aktív közösségi tér is. Ennek a koncepciónak a legfontosabb eleme a beépített "Community Hub", egy modern, valós idejű chat alkalmazás, amely vizuálisan és funkcionálisan is a népszerű gamer és fejlesztői kommunikációs platformokat (mint például a Discord) idézi.

4.8.1 A chat felület felépítése és funkciói

A közösségi modul célja, hogy a tanulók és az oktatók zökkenőmentesen tarthassák a kapcsolatot az adott kurzuson belül. A felület három fő szerkezeti egységből áll:

1. **Csatornaválasztó (Sidebar):** A bal oldali sávban a felhasználók a kurzushoz tartozó különböző szöveges csatornák (Text Channels) között válthatnak (például *#general*, *#help*, *#ui*). Ez lehetővé teszi a témák tematizálását és a strukturált kommunikációt. Itt kapott helyet a tanulási útvonalhoz (Learning Path) visszavezető gyorsgomb is.
2. **Üzenőfal (Messages Section):** A középső, legszélesebb oszlop a tényleges kommunikáció helyszíne. Itt jelennek meg a valós időben frissülő üzenetek, kiegészítve a küldő profilképével, nevével és az időbélyeggel.
3. **Tagok panelje (Members Panel):** A jobb oldali sáv (asztali nézetben) kilistázza a kurzushoz csatlakozott felhasználókat, vizuális visszajelzést adva a közösség méretéről és a résztvevőkről.

4.8.2 Felhasználói élmény (UX) és nézetmódok

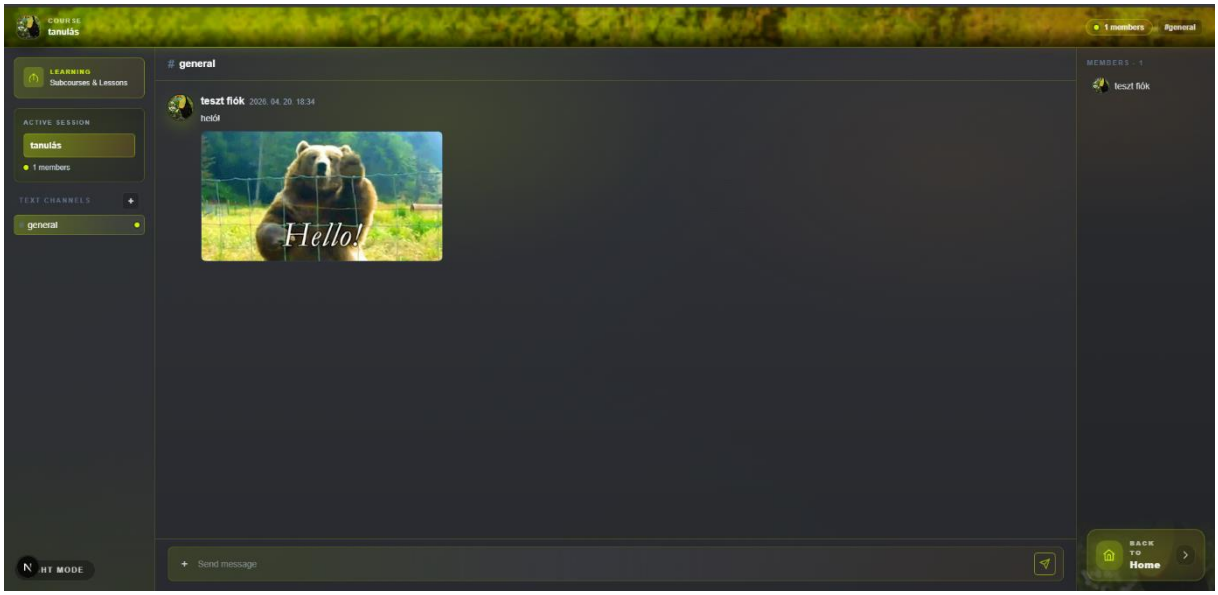
A Community Hub kialakításakor a maximális rugalmasságra törekedtünk. A komponens (ahogy a fullScreen paraméter is mutatja) kétféle módban képes működni:

- **Beágyazott (Kártya) nézet:** Ilyenkor a chat a kurzus oldalának szerves részeként, egy elegáns, árnyékolt dobozban jelenik meg, az adott kurzus egyedi arculati színeit (elsődleges és másodlagos színkódok) használva a fejléceken.
- **Teljes képernyős (Fullscreen) mód:** A fókuszált kommunikációhoz a felhasználók egy dedikált, teljes képernyős nézetet is használhatnak, amely kitölti a böngészőablakot, minimalizálva a zavaró tényezőket.

4.8.3 Integráció a kurzusokkal

A chat modul nem egy globális, mindenki számára elérhető szoba, hanem szigorúan a kurzusokhoz kötődik. A munkamenet-kezelés és az adatbázis-struktúra biztosítja, hogy a felhasználók csak azoknak a kurzusoknak a csatornáit lássák, és csak oda tudjanak üzenetet küldeni, ahová már sikeresen csatlakoztak. Ez a szeparáció garantálja az irreleváns üzenetek (spam) elkerülését és az adott téma iránt érdeklődő mikro-közösségek kialakulását.

Kép a chatszobáról:


COURSE
tanulás
1 members
#general
LEARNING
Subcourses & Lessons
ACTIVE SESSION
tanulás
1 members
TEXT CHANNELS
general
general
teszt fiók 2026. 04. 20. 18:34
helól
Hello!
MEMBERS - 1
teszt fiók
Send message
BACK TO
Home

5 Tesztelés

5.1 Backend tesztelés

Backend teszteléséhez Pest tesztelési keretrendszert alkalmaztuk, a tesztelés során sikeresen leteszteltünk 15 végpontot, a regisztrációt, bejelentkezést és a jogosultságok kezelését.

```
INFO <!-- 15 db végpont teszt -->.


PASS Tests\Unit\RolesTest
✓ it correctly identifies an admin user                          0.10s
✓ it correctly identifies a regular user                         0.02s
"Password in DB is: $2y$04$ElRIep0zCHD.9Vtvfk8uBenLxza2mnWyZaA2/KE7w2uvZB6Sik5f6" // tests\Feature\AuthenticationTest.php:36

PASS Tests\Feature\AuthenticationTest
✓ it can register a new user                                     0.57s
✓ it can login with correct credentials                          0.09s
✓ it cannot login with an incorrect password                     0.23s

PASS Tests\Feature\EndpointTest
✓ it can fetch the users list                                    0.04s
✓ it can fetch the categories list                               0.02s
✓ it can fetch the subscriptions list                            0.02s
✓ it can fetch the text channels list                            0.02s
✓ it can fetch the ct progressions list                          0.02s
✓ it can fetch the achievements list                             0.02s
✓ it can fetch the achievement types list                        0.02s
✓ it can fetch the transactions list                             0.02s
✓ it can fetch the subcourses list                               0.02s
✓ it can fetch the direct chats list                             0.02s
✓ it can fetch the messages list                                 0.02s
✓ it can fetch the message attachments list                      0.02s
✓ it can fetch the emojis list                                   0.02s
✓ it can fetch the feeds list                                    0.02s
✓ it can fetch the posts list                                    0.02s

Tests:    20 passed (21 assertions)
Duration: 1.50s
```

A backend tesztelésére összesen 18 feature és 2 unit tesztet írtunk.

5.2 Frontend Tesztelés

A frontend tesztelést a Cypress keretrendszerrel végeztük, és teszteltük a teljes bejelentkezési folyamatot. Ehhez a már létező JSX kódot kiegészítettük data-cy HTML attribútumokkal, amik segítik a mezők beazonosítását. A teszt során létrehozunk egy véletlenszerű e-mail-el egy felhasználót, majd bejelentkeztetjük. A frontend-be épített hibakezelés megfelelő működését is ellenőriztük.

```
Running:  LoginTest.cy.ts                                                    (1 of 1)

Test the login process
  ✓ Step 1: Register a new account: name, coursecreatenumber, email, pwd, pwdconfirm should exist, take up values and registration should redirect to login,error should NOT be
visible, success should be visible (10097ms)
  ✓ Step 2: Login with the new account: feed the registration values into the login, error should NOT be visible, success should be visible and then redirect to main page (787
5ms)

2 passing (18s)

(Results)

  Tests:        2
  Passing:      2
  Failing:      0
  Pending:      0
  Skipped:      0
  Screenshots:  0
  Video:        false
  Duration:     18 seconds
  Spec Ran:     LoginTest.cy.ts


================================================================================================

(Run Finished)

     Spec                                              Tests  Passing  Failing  Pending  Skipped
  ✓  LoginTest.cy.ts                          00:18        2        2        -        -        -
  ✓  All specs passed!                        00:18        2        2        -        -        -
```

6 Deployment

A projekt felépítésénél fontos szempont volt, hogy a frontendet és a backendet teljesen különválasszuk. Bár az üzemeltetés sokszor egy külön DevOps csapat feladata lenne, a tapasztalatszerzés miatt mi is szerettük volna magunkévá tenni ezt a folyamatot, így a teljes környezetet mi magunk raktuk össze.

6.1 Docker konténerizálás

A projektet Docker segítségével konténerizáltuk. Ezzel elértük, hogy a program pontosan ugyanúgy működjön a fejlesztők gépein, mint az éles szerveren. Ez a megoldás megkönnyíti a költözést egy másik szerverre vagy operációs rendszerre, mivel a környezeti beállítások mindig állandóak maradnak. A rendszerünket három fő konténerre bontottuk:

- **PostgreSQL szerver:** az adatbázis tárolásáért felel.
- **Laravel backend:** a háttérfolyamatokat és az API-t szolgálja ki.
- **Next.js frontend:** a felhasználói felületet és a weboldal megjelenítését kezeli.

6.2 Projekt futtatása lokálisan

A helyi fejlesztői környezet elindítását igyekeztünk a lehető legegyszerűbbre csinálni. Ha teljesen tiszta lappal szeretnénk indulni, érdemes a régi lokális Docker image-eket először törölni. Az indítás a következő lépésekből áll:

1. A docker mappában kiadjuk a konténerek indításáért felelős parancsot: docker compose up
2. Lefuttatjuk a docker-setup.bat fájlt. Ez a szkript automatikusan elvégzi a backend alapvető beállításait, például telepíti a szükséges csomagokat és létrehozza az adatbázis tábláit.
3. Mivel a Laravel keretrendszernek írási jogokra van szüksége a naplófájlok (logok) és a gyorsítótár (cache) kezeléséhez, a terminálban kiadjuk az alábbi két parancsot: chown -R www-data:www-data storage bootstrap/cache chmod -R 775 storage bootstrap/cache

Ezek után a lokális környezet készen áll a használatra.

6.3 AWS S3 és Cloudflare R2 használata

A médiafájlok – például a felhasználók profilképei és a kurzusokhoz tartozó videók – tárolásánál fontos volt, hogy ne a saját backend szerverünket terheljük. Ha ezeket a saját szerverünkön tárolnánk, a rendszer lelassulhatna, és egy újraindítás során könnyen elveszhetnének az adatok. Ennek megelőzésére felhőalapú tárhelyeket (Object Storage) integráltunk a projektbe, méghozzá két különböző szolgáltató bevonásával:

- **Amazon AWS S3 a képekhez:** A statikus képek (például a .jpg, .png, .webp kiterjesztésű profilképek) tárolására a megbízható és iparági standardnak számító AWS S3 szolgáltatást használjuk.
- **Cloudflare R2 a videókhoz:** A videók kiszolgálása jóval nagyobb adatforgalmat és sávszélességet igényel. A költségek és a sebesség optimalizálása érdekében a

nagyobb méretű videófájlokhoz a Cloudflare R2 szolgáltatását választottuk. Ennek óriási előnye, hogy a kimenő adatforgalom (letöltés) teljesen ingyenes, így a videók lejátszása nem generál extra költségeket.

A Next.js weboldalunk a megfelelő felhőtárhelyről, nyilvános linkeken keresztül tölti be ezeket a tartalmakat. Ezzel a megosztott megoldással a szerverünk terhelése minimális marad, a rendszerünk pedig rugalmas és költséghatékony a jövőbeli fejlesztésekhez.

7 Felhasználói útmutató

A Skilltree platform kipróbálásához és a funkciók értékeléséhez a rendszer (adatbázis seeding segítségével) automatikusan létrehoz két alapértelmezett tesztfiókot. Ezekkel a hitelesítő adatokkal a telepítés után azonnal be lehet jelentkezni a felületen, így a rendszer minden jogosultsági szintje egyszerűen tesztelhető.
A teszteléshez az alábbi hozzáférések használhatók:
1. **Adminisztrátor fiók (Rendszergazda)** Ez a fiók teljes körű hozzáféréssel rendelkezik, beleértve az adminisztrációs felületet (Admin Panel), a felhasználók és a kurzusok moderálását.

- E-mail cím: ad@m.in
- Jelszó: admin123

2. **Általános Teszt Felhasználó (Tanuló / Oktató)** Ezzel a fiókkal a normál felhasználói élmény (kurzusok böngészése, csatlakozás, chatelés) tesztelhető.

- E-mail cím: us@e.r
- Jelszó: useruser

A weboldal ezen az url-en elérhető: http://skillstree-vizsgaremek-frontend.jcloud.jedlik.cloud

8 Irodalomjegyzék

Next.js. React alapú frontend keretrendszer. url: https://nextjs.org/

React. JavaScript könyvtár felhasználói felületek készítéséhez. url: https://react.dev/

Laravel 12. Backend PHP keretrendszer. url: https://laravel.com/docs/12.x

FakerPHP. Tesztadat-generáló könyvtár PHP-hoz. url: https://fakerphp.org/

Docker. Konténerizációs platform alkalmazások futtatásához. url: https://www.docker.com/

Amazon S3. Skálázható felhőalapú objektumtároló (Simple Storage Service). url: https://aws.amazon.com/s3/

Cloudflare R2. S3-kompatibilis felhőalapú objektumtároló szolgáltatás. url: https://developers.cloudflare.com/r2/